Algorithms

DYNAMIC PROGRAMMING

Last Week

- Flow

This Week

- Dynamic Programming

DP Algorithm

General structure of an algorithm based on DP:

- Define the table.
- Identify base cases (sub-problems that can be filled directly).
- Write a recursive formula, and then, instead of performing the recursion, fill the table.
- Conclude the value of the solution from the table.
- Retrieve the solution.

Exercise 1

Let $a = (a_1, \ldots, a_n)$ be a sequence of numbers. A subsequence b of a is any subsequence, not necessarily consecutive. Find the length of a longest monotone non-decreasing subsequence of a (the optimal subsequence isn't necessarily unique).

Example:

a = (1, 2, 6, 9, 1, 2, 3, 2, 4, 8, 9)

Exercise 1

Let $a = (a_1, \ldots, a_n)$ be a sequence of numbers. A subsequence b of a is any subsequence, not necessarily consecutive. Find the length of a longest monotone non-decreasing subsequence of a (the optimal subsequence isn't necessarily unique).

Example:

a = (1, 2, 6, 9, 1, 2, 3, 2, 4, 8, 9)

a = (1, 2, 6, 9, 1, 2, 3, 2, 4, 8, 9)

The length is 7.

Exercise 1 - Solution

- Let $B[i]$ be the length of the maximal monotone non decreasing subsequence that starts on the index i.
- For $i = n, \dots, 1$:

$$B[i] = \begin{cases} 1 & , \quad i = n \\ 1 + \max\limits_{j:j>i}\{B[j] \mid a_i \leq a_j\} & , \quad 1 \leq i < n \end{cases}$$

Base case

The longest subsequence that begins after index i and can be concatenated after a_i

- Return $\max\limits_{1 \leq i \leq n}\{B[i]\}$

Exercise 1 - Solution

a = (1, 2, 6, 9, 1, 2, 3, 2, 4, 8, 9)

B = (_, _, _, _, _, _, _, _, _, _, _)

$$B[i] = \begin{cases} 1, & i = n \\ 1 + \max\limits_{j:j>i}\{B[j] \mid a_i \le a_j\}, & 1 \le i < n \end{cases}$$

Exercise 1 - Solution

a = (1, 2, 6, 9, 1, 2, 3, 2, 4, 8, 9)

B = (_, _, _, _, _, _, _, _, _, _, 1)

$$B[i] = \begin{cases} 1, & i = n \\ 1 + \max\limits_{j:j>i}\{B[j] \mid a_i \le a_j\}, & 1 \le i < n \end{cases}$$

Exercise 1 - Solution

a = (1, 2, 6, 9, 1, 2, 3, 2, 4, 8, 9)

B = (_, _, _, _, _, _, _, _, _, _, 1)

$$B[i] = \begin{cases} 1, & i = n \\ 1 + \max\limits_{j:j>i}\{B[j] \mid a_i \le a_j\}, & 1 \le i < n \end{cases}$$

Exercise 1 - Solution

a = (1, 2, 6, 9, 1, 2, 3, 2, 4, 8, 9)

B = (_, _, _, _, _, _, _, _, _, 2, 1)

$$B[i] = \begin{cases} 1, & i = n \\ 1 + \max\limits_{j:j>i}\{B[j] \mid a_i \le a_j\}, & 1 \le i < n \end{cases}$$

Exercise 1 - Solution

a = (1, 2, 6, 9, 1, 2, 3, 2, 4, 8, 9)

B = (_, _, _, _, _, _, _, _, _, 2, 1)

$$B[i] = \begin{cases} 1, & i = n \\ 1 + \max\limits_{j:j>i}\{B[j] \mid a_i \le a_j\}, & 1 \le i < n \end{cases}$$

Exercise 1 - Solution

a = (1, 2, 6, 9, 1, 2, 3, 2, 4, 8, 9)

B = (_, _, _, _, _, _, _, _, 3, 2, 1)

• • •

B = (7, 6, 3, 2, 6, 5, 4, 4, 3, 2, 1)

Return $\max_{1 \le i \le n} \{B[i]\}$

Exercise 1 - Solution

a = (1, 2, 6, 9, 1, 2, 3, 2, 4, 8, 9)

B = (_, _, _, _, _, _, _, _, 3, 2, 1)

• • •

B = (**7**, 6, 3, 2, 6, 5, 4, 4, 3, 2, 1)

Return $\max_{1 \le i \le n} \{B[i]\}$

Exercise 1 - Solution

Complexity:

n elements in the array, up to n operations for the max for each element – $O(n^2)$

Exercise 2 - Intro

Let $A_{l\times m}, B_{m\times n}, AB = C_{l\times n}$:

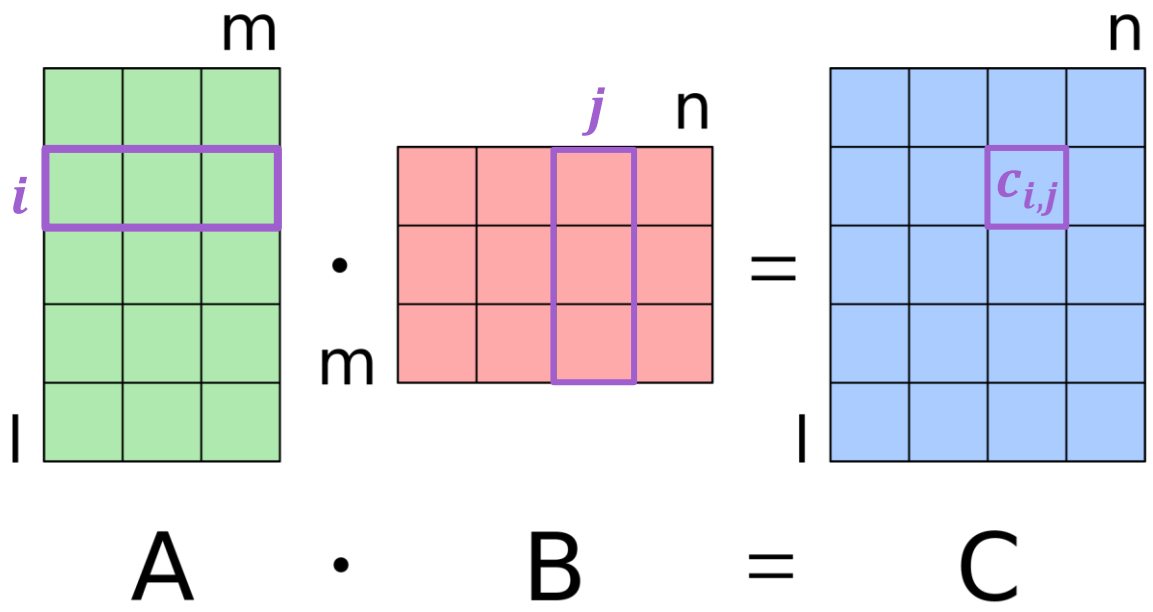
Each element in C:

$c_{i,j} = A_i B^j$

(i-th row in A, j-th column in B)

Calculating single element: $O(m)$

All together: $O(mln)$



Exercise 2

Let $A_1, \ldots, A_n$ be n matrices such that A_i is a matrix of size $m_i \times m_{i+1}$ and the product $A_1 A_2 \ldots A_n$ is defined. Notice that the number of operations to compute the multiplication of the matrices depends on the order of the product.

Suggest an algorithm that finds the minimal number of multiplications required to compute the product $A_1 A_2 \ldots A_n$.

Exercise 2

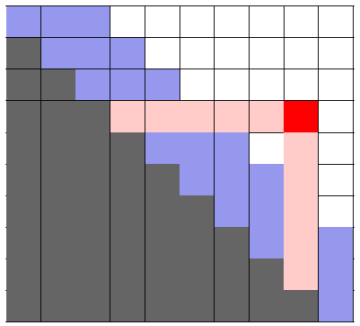
Example: Suppose A_1 is 1×3 , A_2 is 3×5 and A_3 is 5×6.

Computing $(A_1 A_2)\, A_3$ takes $(1 \cdot 3 \cdot 5) + (1 \cdot 5 \cdot 6) = 45$ multiplications.

Computing $A_1 (A_2 A_3)$ takes $(3 \cdot 5 \cdot 6) + (1 \cdot 3 \cdot 6) = 108$ multiplications.

Exercise 2 - Solution

- Table: Let $B[i,j]$ be the minimal number of multiplications required to compute the product $A_i \dots A_j$.



- Base case: $B[i,i] = 0$ (diagonal).
- For $i,j = 1,\dots,n$:

$$B[i,j] = \begin{cases} 0 & ,i = j \\ \min\limits_{i<k\leq j}\{B[i,k-1] + B[k,j] + m_i \cdot m_k \cdot m_{j+1}\}, & i < j \end{cases}$$

- Return $B[1,n]$.

Exercise 2 - Solution

Consider the following six matrix problem:

Matrix	A_1	A_2	A_3	A_4	A_5	A_6
Dimensions	10x20	20x5	5x15	15x50	50x10	10x15

Initialize the table:

i\j	1	2	3	4	5	6
1	0					
2		0				
3			0			
4				0		
5					0	
6						0

Exercise 2 - Solution

Fill first diagonal $B[i, i+1] = m_i \cdot m_{i+1} \cdot m_{i+2}$:

- B[1, 2] = 10x20x5 = 1000
- B[2, 3] = 20x5x15 = 1500
- B[3, 4] = 5x15x50 = 3750
- B[4, 5] = 15x50x10 = 7500
- B[5, 6] = 50x10x15 = 7500

Matrix	A_1	A_2	A_3	A_4	A_5	A_6
Dimensions	10x20	20x5	5x15	15x50	50x10	10x15

i\j	1	2	3	4	5	6
1	0	1000				
2		0	1500			
3			0	3750		
4				0	7500	
5					0	7500
6						0

Exercise 2 - Solution

Fill chains of length 3:

Matrix	A_1	A_2	A_3	A_4	A_5	A_6
Dimensions	10x20	20x5	5x15	15x50	50x10	10x15

$B[1,3]=\min\{(B[1,1]+B[2,3]+m_1m_2m_4),$
$(B[1,2]+B[3,3]+m_1m_3m_4)\}$
$=\min\{(0+1500+10x20x15),$
$(1000+0+10x5x15)\}$
$=\min\{4500, 1750\} = 1750$

i\j	1	2	3	4	5	6
1	0	1000	1750			
2		0	1500	8750		
3			0	3750	6250	
4				0	7500	9750
5					0	7500
6						0

Exercise 2 - Solution

Fill the rest:

Matrix	A_1	A_2	A_3	A_4	A_5	A_6
Dimensions	10x20	20x5	5x15	15x50	50x10	10x15

i\j	1	2	3	4	5	6
1	0	1000	1750	7250	7750	**8750**
2		0	1500	8750	7250	8500
3			0	3750	6250	7000
4				0	7500	9750
5					0	7500
6						0

Exercise 2 - Solution

Complexity:

- Need to fill $O(n^2)$ cells in the table.
- Each cell takes $O(n)$ (finding the minimum).
- Total running time: $O(n^3)$.

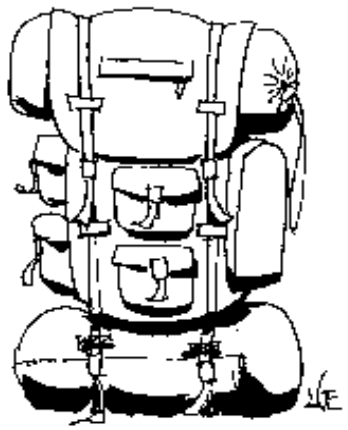
Exercise 3 – The Knapsack Problem

Consider a bag with integer capacity W and n items, where each item i has integer positive weight $w_i > 0$ and value v_i.

Suggest an algorithm that finds a subset of the elements such that the profit (the sum of values of the elements in the subset) is maximal and the overall weight (the sum of weights of the elements in the subset) is not more than W.

Exercise 3 – Example

Item #	Weight	Value
1	1	8
2	3	6
3	5	5
4	4	6



W=8

A possible packing: Items 2 and 3. Value: 11

An optimal packing: Items 1,2,4. Value: 20

Exercise 3 - Solution

- For $0 \leq i \leq n, 0 \leq w \leq W$, let $B[i, w]$ be the maximal profit we can get from a subset of the first i items and a knapsack with capacity w.
- Base cases:

$$w = 0 : B[i, 0] = 0, \forall i$$
$$i = 0 : B[0, w] = 0, \forall w$$

Exercise 3 - Solution

- For $1 \leq i \leq n, 1 \leq w \leq W$, consider the following options:
  1. Don't take item i - profit and weight left remains the same.
  2. Take item i - only possible if we have enough space, namely $w - w_i \geq 0$. The profit will grow with v_i, but available space will subtract to $w - w_i$.
- Formally:

$$B[i, w] = \max\{v_i + B[i-1, w-w_i], B[i-1, w]\}$$

- Return $B[n, W]$.

Exercise 3 - Solution

Complexity:

- Need to fill $O(nW)$ cells in the table.
- Each cell takes $O(1)$.
- Total running time: $O(nW)$.